

# CALLNGRAM: Architecture and Methodology

## Customer Call Language Analytics via N-gram Language Modeling

Technical Architecture, Probabilistic Formulation, and Experimental Methodology

### 1. Problem Statement

Modern customer support contact centers handle thousands of spoken interactions daily across telecommunications, banking, retail, and tech support domains. Automated quality assurance, agent assist, and query routing require rigorous quantification of spoken language predictability and the discovery of recurring conversational multi-word phrases. While deep neural language models offer high representational capacity, their opaque probability structures, heavy compute requirements, and susceptibility to hallucination make classical statistical N-gram language models essential for transparent, computationally efficient, and mathematically grounded audio dialogue analytics.

### 2. Project Objective

The primary objectives of the CALLNGRAM mini-project are:

- **N-Gram Modeling:** Formulate and implement Unigram (N=1), Bigram (N=2), and Trigram (N=3) language models.
- **Smoothing:** Implement Laplace (Add-1) smoothing for both unconditional and conditional distributions to resolve the zero-frequency problem.
- **Perplexity Scoring:** Evaluate intrinsic dialogue predictability via test-set perplexity on held-out call-center transcripts.
- **Customer Issue Mining:** Extract category- and intent-specific recurrent multi-word phrases from customer service inquiries.
- **Interactive Dashboard:** Deliver a professional Streamlit web application supporting live transcript scoring and data exploration.

### 3. Dataset Description

To ensure academic rigor and linguistic integrity, CALLNGRAM utilizes two distinct, unmerged corpora:

| Property      | Primary: AppTek Dialogues                                               | Secondary: Bitext Support                                            |
|---------------|-------------------------------------------------------------------------|----------------------------------------------------------------------|
| Nature        | Conversational call-center-style transcripts with agent/customer roles  | Customer support query utterances labeled with intent & category     |
| Volume        | 1,746 dialogues (873 customer, 873 agent)                               | 26,872 query utterances                                              |
| Total Tokens  | 1,274,670 tokens (~730.0 words/call)                                    | ~320,000 tokens (~12 words/query)                                    |
| Vocabulary    | 17,868 unique word types                                                | ~8,500 unique word types                                             |
| Diversity     | 16 domains, 14 accent groups                                            | 11 categories, 27 intents                                            |
| Pipeline Role | Core language modeling, Add-1 smoothing, held-out perplexity evaluation | Customer issue phrase mining & category/intent terminology profiling |

*Academic Note:* Neither dataset represents wiretapped or illicit customer audio recordings. AppTek provides benchmark conversational transcripts recorded in call-center settings, while Bitext provides customer service intent texts. The corpora are strictly maintained as separate analytic streams.

## 4. System Architecture

The modular pipeline consists of four decoupled layers: Preprocessing, Statistical Modeling, Evaluation & Analytics, and Interactive Presentation. Precomputed artifacts guarantee deterministic zero-latency startup on Streamlit Cloud.

## 5. Preprocessing and Spoken Marker Preservation

Spoken conversational dialogues feature filled pauses and hesitation markers that carry vital discourse information. In contrast to standard NLP pipelines that strip all non-lexical tokens, CALLNGRAM explicitly preserves spoken discourse markers:

- **Preserved Markers:** *uh, um, hmm, hm, okay, ok, yeah, yep, mhm, ah, oh, woah.*
- **Bracket Normalization:** Annotation artifacts like (uh) and (um,) are cleaned to canonical lexical tokens uh and um.
- **Normalization:** Case lowercasing, Unicode apostrophe/quote standardization, whitespace collapse.
- **Sentence Segmentation:** NLTK sent\_tokenize with automated regex fallback for zero-dependency offline environments.

## 6. Statistical N-gram Language Modeling

Under the Markov assumption, the probability of a word sequence  $W = (w_1, w_2, \dots, w_m)$  is approximated by conditioning each word on only its preceding N-1 history tokens:

- **Unigram (N=1):** Assumes complete word independence:  $P(W) = \prod P(w_i)$
- **Bigram (N=2):** Conditions on the immediately preceding token:  $P(W) = \prod P(w_i | w_{i-1})$
- **Trigram (N=3):** Conditions on two preceding tokens:  $P(W) = \prod P(w_i | w_{i-2}, w_{i-1})$

## 7 & 8. Laplace (Add-1) Smoothing and Probability Formulas

To eliminate zero probabilities for unseen N-grams while maintaining mathematically valid probability distributions, Laplace smoothing adds a pseudo-count of 1 to every transition in the parameter space:

| Model   | Maximum Likelihood Estimate (MLE)                                                                  | Laplace (Add-1) Smoothed Formula                                                                                 |
|---------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Unigram | $P(w) = \text{count}(w) / N$                                                                       | $P(w) = (\text{count}(w) + 1) / (N +  V )$                                                                       |
| Bigram  | $P(w_i   w_{i-1}) = \text{count}(w_{i-1}, w_i) / \text{count}(w_{i-1})$                            | $P(w_i   w_{i-1}) = (\text{count}(w_{i-1}, w_i) + 1) / (\text{count}(w_{i-1}) +  V )$                            |
| Trigram | $P(w_i   w_{i-2}, w_{i-1}) = \text{count}(w_{i-2}, w_{i-1}, w_i) / \text{count}(w_{i-2}, w_{i-1})$ | $P(w_i   w_{i-2}, w_{i-1}) = (\text{count}(w_{i-2}, w_{i-1}, w_i) + 1) / (\text{count}(w_{i-2}, w_{i-1}) +  V )$ |

Where N is total training tokens ( $N = 1,056,726$ ), and  $|V|$  is the fixed vocabulary size ( $|V| = 15,800$  unique words observed during training, plus sentence-terminal token  $\langle s \rangle$ ). This guarantees  $\sum P(w | \text{ctx}) = 1.0$ .

## 9. Perplexity Metric Formulation

Perplexity (PP) measures the effective branching factor of the language model over an evaluated sequence of M tokens:

$$PP(W) = \exp \left( - \frac{1}{M} \sum \ln P(w_i | \text{context}) \right)$$

**Critical Evaluation Constraint:** Perplexity for an input transcript MUST be computed strictly with respect to the reference model trained on the held-out training partition. Computing perplexity from an uploaded transcript's own frequencies is mathematically invalid (trivial self-overfitting).

## 10. Train / Test Partitioning Protocol

- **Split Ratio:** 80% Training (1396 transcripts) / 20% Held-Out Testing (350 transcripts).
- **Seed:** Fixed pseudo-random seed `random_state=42` for exact reproducibility.
- **Zero Leakage:** Whole dialogue transcripts are assigned to partitions. Sentences from the same dialogue are never split across train and test.

## 11. Customer Issue Phrase Intelligence

Beyond statistical n-grams, CALLNGRAM extracts domain- and intent-specific terminology from the Bitext dataset across 11 categories and 27 intents. *Important:* High frequency indicates conversational recurring terminology, not intent classification by itself.

## 12. Streamlit Web Application Workflow

The web application (`app.py`) provides 5 dedicated interactive modules: Dashboard (corpus KPIs), Analyze Transcript (live text/upload scoring against reference models), N-gram Explorer (frequency/probability tables and charts), Issue Intelligence (intent/category drill-downs), and Model Evaluation (comparative academic tables and plots).